

8.1

Given pseudo-code:

```
static boolean flags = new boolean[2]; // initially zero

public void foo() {
    int i = ThreadId.get();           // F.1
    int j = 1 - i;                     // F.2
    while (true) {                     // F.3
        flags[i] = true;               // i would like to enter // F.4
        if (flags[j] == false) {      // you don't             // F.5
            if (flags[i] == true) {    // my request was not reset // F.6
                break;                // i win                  // F.7
            }
        } else {
            // looks like we have a contention
            flags[i] = false; // retreat // F.8
            flags[j] = false; // forcibly reset competitor // F.9
        }
    }
}
```

Consider the case of an evil scheduler that created such an execution that both A.F.4 and B.F.4 preceded A.F.8 and B.F.8, then again A.F.4 and B.F.4 happened in sequence, and so on and so forth. In this case neither A nor B will be able to finish execution of `foo`, because both of them are stuck in the loop. Thus `foo` is not wait-free. `foo` is not lock-free either because of that example. No matter how far into this trace we look, neither thread will be able to return.

The above example also shows that the algorithm is not deadlock-free: threads wait for each other to stop setting their flags and because of everlasting contention they fail to do so. And because starvation freedom implies deadlock freedom, the lack of deadlock freedom implies lack of starvation freedom.

Let's prove that `foo` is obstruction-free. Because threads A and B are equal, let's assume B is frozen in some point of execution. Exact point is irrelevant, because it does not directly influence A's execution. The only thing that matters is the value of `flags` and current position of thread A.

- `flags == (1, *)` and A is in F.6: straight to F.7
- `flags == (1, 0)`: if A starts in F.8 or F.9, it will make a backwards jump to F.4, after which `flags` will be in the same state, and proceed to return. If A starts from any other position, it is a straight pass to F.7
- `flags == (0, 0)`: A will have to do at most one additional iteration to set its flag, and then proceed to return
- `flags == (0, 1)`: A will make a single additional iteration no matter where it starts (except for F.7)
- `flags == (1, 1)` and A is not in F.6: A will make an additional iteration to set the flags, then return.